

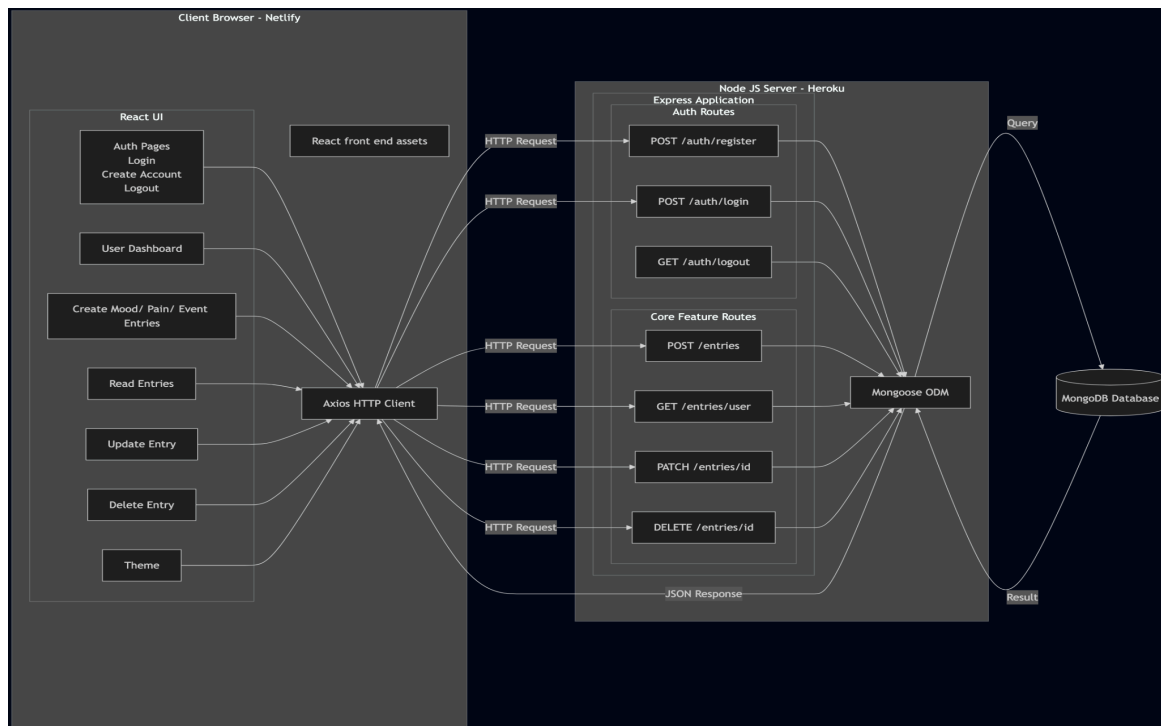
How The Village uses Client/Server Architecture

Overview

The Village is built using a client/server architecture typical of the MERN stack. The client side consists of a React single page application (SPA) hosted on Netlify, which runs in the user's web browser. This frontend is responsible for rendering the user interface, managing client-side interactions, and sending HTTP requests to the backend when data needs to be created, retrieved, or updated.

The server-side is a Node.js application hosted on Heroku and built using the Express framework. The Express server handles incoming requests from the React client, processes application logic, and exposes RESTful API endpoints. Routes are organised into groups, with authentication-related routes handled separately from core feature routes to keep the backend structure modular and maintainable.

Communication between the client and server occurs through HTTP requests sent using the Axios HTTP client in the React application. The server then interacts with the database through Mongoose, an Object Data Modelling (ODM) library that provides a structured way to define schemas and manage data in MongoDB. This layered architecture separates the user interface, application logic, and data storage, making the system easier to develop, maintain, and scale.



Client/ Server Communication

The React client (hosted on Netlify) communicates with the Express API (hosted on Heroku) over HTTPS using Axios. The API exposes RESTful endpoints grouped by purpose (e.g. `/api/v1/auth/*` for authentication, `/api/v1/entries/*` for mood/ pain/ event CRUD), and the client uses the standard HTTP verbs - `POST` for create, `GET` for read, `PATCH` for updates, and `DELETE` for removals. We will keep a single Axios instance configured from an environment variable (e.g. `REACT_APP_API_URL`) so builds point to the correct Heroku URL; the server will enable CORS for the Netlify origin and use clear status codes and JSON payloads (`Content-Type: application/json`) with a simple response envelope like `{ success, data, error }`.

On our client, we'll create an Axios instance with sensible defaults (baseUrl, timeout) and interceptors to attach auth tokens to `Authorization` headers and to handle common error cases centrally. Implement standard retry/backoff or user-facing error messages for network failures, and will consider optimistic UI patterns for quick PATCH/DELETE interactions with rollback on server error. On the server, validate and sanitise incoming JSON, use Mongoose schemas to shape responses, and never return sensitive fields (e.g. hashed passwords). Finally, we will wire deployment and runtime settings through environment variables (API URL, JWT secret, DB URI) so local, staging and production environments remain consistent and secure.

Data Distribution

We will treat the Express API / MongoDB as the single source of truth: the server owns persistent data and the client requests exactly what it needs via JSON REST endpoints (e.g. `GET /api/v1/entries?type=mood&limit=20&page=2`). Our app may implement server-side pagination (if necessary), filtering and field projection (Mongoose `limit()`, `skip()`, and `select()` / `.lean()`) so responses are compact and predictable, and use MongoDB indexes on common query fields (userId, createdAt, entry type) and aggregate pipelines for summary queries (weekly mood averages) to keep response times low. On the server, we'll map database documents to clear response DTOs (data transfer objects) that exclude sensitive fields (never sending hashed passwords or internal tokens).

On the client, we will be keeping a minimal local cache of fetched data (React state, Context or a small library like React Query) and prefer revalidation on focus or after mutations. For mutation workflows (create/ patch/ delete), send concise requests (IDs and changed fields) and use optimistic updates with rollback on error to keep the UI snappy. Caching critical read responses and queue writes in IndexedDB/localStorage while the service worker is offline, then sync them when connectivity returns, for offline resilience. Finally, we'll document and version the API (simple OpenAPI/markdown) so frontend and backend teams - or future markers - understand the shape and access patterns for each resource.

Data Security

The Village implements end-to-end protections: all client/server traffic uses HTTPS and TLS, and our API will enforce secure headers via Helmet and well-configured CORS for the Netlify origin. User credentials are hashed using Node's `cryptojs` on the server, and authentication uses short-lived JWTs with a refresh-token pattern; tokens are sent in `Authorization: Bearer <token>` headers on every request. Secrets and configuration (JWT secret, DB URI) are kept out of source control and managed with Heroku and Netlify environment variables; locally we'll access them via Node's `process.env` (set in the shell or via platform CLIs). We do not commit secrets in the repository.

On the backend we enforce input validation and sanitisation (Mongoose schema validation plus libraries like `express-validator`) to prevent injection and malformed data, and apply rate limiting (e.g. `express-rate-limit`) to limit abuse. Data at rest is protected by the database provider (enable encryption at rest and least-privilege DB users in MongoDB Atlas), regular backups, and minimal exposure of sensitive fields in API responses. We log auth events and errors with concise, privacy-aware logs for incident review, and schedule periodic security checks and dependency updates to keep our stack resilient.

Feature Distribution

The Village splits responsibilities so the client (React on Netlify) handles all user-facing behaviour: rendering lists and forms for mood/ pain/ event entries, client-side validation and accessibility affordances, optimistic UI updates for PATCH/DELETE flows, and local theme toggling (dark/ light) with immediate UI persistence. The frontend uses Axios to call the API for full CRUD operations - `POST /api/v1/entries`, `GET /api/v1/entries`, `PATCH /api/v1/entries/:id`, `DELETE /api/v1/entries/:id` - and keeps a minimal local cache (React state/Context or a light cache library) that revalidates after mutations. Theme changes are applied instantly in the UI and persisted with a concise `PATCH /api/v1/users/:id/settings` request so settings survive across devices.

The server (Express on Heroku) owns business logic, persistence and access control: routes are organised into `auth` (login, logout, refresh) and `core` (entries, user settings) groups, backed by Mongoose models for `User` and `Entry` (type: mood|pain|event). The API enforces validation, ownership checks (only the entry owner can PATCH/DELETE), pagination and field projection for list endpoints, and returns DTOs that exclude sensitive fields. Authentication is checked via `Authorization: Bearer <token>` on every protected route; the server also handles createdAt/ updatedAt, optional soft-deletes for safety, and concise audit logging for important events. This separation keeps the client lightweight and responsive while ensuring the server remains the single source of truth for data integrity.

Authorisation

The Village implements token-based authorisation: the server issues short-lived JWT access tokens (signed with our JWT secret) and longer-lived refresh tokens tracked server-side for revocation/ rotation. On the backend we use the `jsonwebtoken` library and an Express `verifyToken` middleware that validates tokens, attaches `req.user` (id and scopes/role), and rejects invalid or expired tokens. Route-level guards (`requireRole('admin')`, `requireScope('entries:write')`) enforce role or scope checks, and ownership is enforced at the DB layer by always filtering updates/deletes by `userId` (e.g. `Entry.findOneAndUpdate({ _id, userId: req.user.id }, ...)`) rather than trusting client input. This keeps the principle of least privilege, supports simple RBAC (role-based access control), and makes token revocation (via a refresh-token store or blacklist) possible if credentials must be invalidated.

On the client we attach `Authorization: Bearer <token>` using an Axios instance and interceptors; a React Router private-route pattern prevents unauthenticated routes from rendering. The Axios interceptor handles 401s by calling `/api/v1/auth/refresh` to rotate tokens and retry the original request. For our project, we may persist refresh tokens in `localStorage` (with short access-token lifetimes) but we mitigate XSS (cross-site-scripting) risk through CSP (content security policy), input sanitisation and minimal use of `innerHTML`. We also add automated tests (Jest for client guards and supertest server middleware) and concise audit logs for login, token refresh and denied access to help detect and investigate abnormal authorisation events.

Validation

The Village implements server-side validation using `express-validator` middleware in our Express API so every incoming request is checked before reaching business logic or the database. Validation chains (e.g. `body()`, `param()`, `query()` rules) run as route middleware and results are collected with `validationResult()`; if checks fail the server responds with a consistent `400` payload describing the errors, preventing invalid or malicious data from progressing. This middleware approach keeps validation colocated with routes while remaining reusable across endpoints.

We centralise rules by exporting small validator modules per resource (e.g. `registerValidator`, `entryCreateValidator`) and prefer `checkSchema` or chained validators for complex inputs so rules are explicit and testable. Sanitisation helpers are applied where needed and Mongoose schema validation (`runValidators: true`) is used as a final safety net before writes. Test coverage (Jest and supertest) targets bad payloads and edge cases, and the server normalises errors into a simple `{ success: false, errors: [...] }` structure so the client can surface friendly messages and the backend remains auditable and robust.

References

Axios. (n.d.). *Axios documentation*. Retrieved March 4, 2026, from <https://axios-http.com/docs/intro>

Express.js. (n.d.). *Express documentation*. Retrieved March 4, 2026, from <https://expressjs.com/>

MongoDB Inc. (n.d.). *MongoDB documentation*. Retrieved March 1, 2026, from <https://www.mongodb.com/docs/>

Mongoose. (n.d.). *Mongoose ODM documentation*. Retrieved March 1, 2026, from <https://mongoosejs.com/docs/>

Node.js Foundation. (n.d.). *Node.js documentation*. Retrieved March 4, 2026, from <https://nodejs.org/en/docs>

React Team. (n.d.). *React documentation*. Retrieved March 7, 2026, from <https://react.dev/learn>

Auth0. (n.d.). *JSON Web Token introduction*. Retrieved March 1, 2026, from <https://jwt.io/introduction>

Authgear. (n.d.). *Cryptographic failures and secure password storage*. Retrieved March 7, 2026. <https://www.authgear.com/post/cryptographic-failures-owasp>

Node Security. (n.d.). *Cryptographic recommendation summary*. Retrieved March 7, 2026. <https://node-security.com/posts/cryptographic-recommendation-summary/>

WorkOS. (2026). *Developer's guide to strong passwords*. Retrieved March 7, 2026. <https://workos.com/blog/developers-guide-strong-passwords>